

CSA 1718 – Artificial Intelligence

Name : P. Yaswanth

Reg No : 192524167

Experiment: Prolog Program for Planet Database

Aim

To write and execute a Prolog program to create a database containing information about **planets** and retrieve the information using Prolog queries.

Objective

- To understand how facts are represented in Prolog.
- To create a simple database of planets.
- To retrieve planet information using queries.

Theory

In Prolog, a database can be created using **facts**. Each planet can be represented using properties such as its name, type, and position from the Sun.

Prolog Program

% Planet Database

planet(mercury, terrestrial, 1).

planet(venus, terrestrial, 2).

planet(earth, terrestrial, 3).

planet(mars, terrestrial, 4).

planet(jupiter, gas_giant, 5).

planet(saturn, gas_giant, 6).

planet(uranus, ice_giant, 7).

planet(neptune, ice_giant, 8).

% Rule to find terrestrial planets

terrestrial_planet(Planet) :-

planet(Planet, terrestrial, _).

% Rule to find gas giant planets

gas_giant_planet(Planet) :-

planet(Planet, gas_giant, _).

% Rule to find ice giant planets

ice_giant_planet(Planet) :-

planet(Planet, ice_giant, _).

Sample Queries

1. Find all planets

?- planet(Planet, Type, Position).

Output

Planet = mercury,

Type = terrestrial,

Position = 1 ;

Planet = venus,

Type = terrestrial,

Position = 2 ;

Planet = earth,

Type = terrestrial,

Position = 3 ;

Planet = mars,
Type = terrestrial,
Position = 4 ;

Planet = jupiter,
Type = gas_giant,
Position = 5 ;

Planet = saturn,
Type = gas_giant,
Position = 6 ;

Planet = uranus,
Type = ice_giant,
Position = 7 ;

Planet = neptune,
Type = ice_giant,
Position = 8.

2. Find the type of Earth

?- planet(earth, Type, Position).

Output

Type = terrestrial,
Position = 3.

3. Find all terrestrial planets

?- terrestrial_planet(Planet).

Output

Planet = mercury ;

Planet = venus ;

Planet = earth ;

Planet = mars.

4. Find all gas giant planets

?- gas_giant_planet(Planet).

Output

Planet = jupiter ;

Planet = saturn.

5. Find the position of Neptune

?- planet(neptune, Type, Position).

Output

Type = ice_giant,

Position = 8.

Algorithm

1. Start the program.
2. Define facts for the eight planets.
3. Store the planet name, type, and position from the Sun.
4. Define rules to identify different types of planets.
5. Enter a query.
6. Prolog searches the database for matching facts.
7. Display the result.
8. Stop.

Out Put

```
1 % Planets Database
2
3 planet(mercury).
4 planet(venus).
5 planet(earth).
6 planet(mars).
7 planet(jupiter).
8 planet(saturn).
9 planet(uranus).
10 planet(neptune).
11
12 position(mercury, 1).
13 position(venus, 2).
14 position(earth, 3).
15 position(mars, 4).
16 position(jupiter, 5).
17 position(saturn, 6).
18 position(uranus, 7).
19 position(neptune, 8).
20
21 planet_position(Planet, Position) :-
22     position(Planet, Position).
```

```
?- sum_to_n(5, Sum).
procedure `(>A)` does not exist
knows(ravi, machine_learning).
Sum of natural numbers = 55
procedure `knows(A,B)` does not exist
dob(lohith, DOB).
DOB = date('15,8,2005)
student_teacher(lohith, Teacher).
Teacher = revathi
planet_position(earth, Position).
Position = 3

?- planet_position(earth, Position).
```

Result

The Prolog program was successfully executed to create a **Planet Database** and retrieve information about planets using Prolog facts, rules, and queries.